
XFChess Smart Contract Audit Specification

Program: xfchess-game

Repository: <https://github.com/trilltino/XFChess>

Commit: d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd

Program ID: 8tevgspityTTG45KvvRtWV4GZ2kuGDBYWMXouFGquyDU

Cluster: Solana devnet (pinned Anchor provider)

Prepared: 13 September 2026

Prepared for external smart-contract security review. This document is a specification and audit-preparation artifact, not a security audit opinion.

Contents

01. Revision since the prior audit package	02. Executive summary
03. Scope and evidence basis	04. Definitions and trust model
05. Privileged roles and authorities matrix	06. Build, cluster and deployment controls
07. Dependency and trust map	08. PDA and account architecture
09. Core state layouts	10. Game state machine
11. Game instruction matrix	12. Move validation and anti-replay
13. Terminal game semantics	14. Game escrow and SOL lifecycle
15. Settlement accounting equations	16. Tournament state machine
17. Tournament initialization and capacity	18. Tournament registration and entry economics
19. Tournament prize lifecycle	20. Swiss architecture
21. Single-elimination architecture	22. Tournament session-key architecture
23. Global persistent session architecture	24. Delegation lifecycle and MagicBlock trust boundary
25. ER hot-path write boundary	26. ER unavailability and forced recovery
27. Dispute model	28. Profiles, usernames, external ELO
29. ELO and financial profile settlement	30. Treasury and fee architecture
31. Time, crank, and timeout semantics	32. Error surface and rejection taxonomy
33. Security invariants for an external auditor	34. Threat model
35. Known trade-offs and residual trust assumptions	36. Audit focus areas / suggested test matrix
37. Static source review checklist	38. Test-suite evidence and auditability
39. Recommended end-to-end audit scenarios	40. Deployment handoff checklist
41. Code reference index	42. Appendix A - Exact constants relevant to audit
43. Appendix B - Review questions for the audit kickoff	

XFChess Smart Contract Audit Specification / Architecture Document

AUDIT PACKAGE

PINNED SOURCE

DEVNET TARGET

Program: xfchess-game **Repository:** <https://github.com/trilltino/XFChess> **Commit:** d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd **Cluster:** Solana devnet

Program ID: 8tevgspityTTG45KvvRtWV4GZ2kuGDBYWMXouFGquyDU

Prepared: 13 September 2026 **Purpose:** source-of-truth architecture and intended-behavior package for an external smart-contract security audit.

AUDIT-STATUS DISCLAIMER

This document is an architecture/specification package, not an audit opinion. It describes behavior observed in the pinned source and identifies security assumptions, required invariants, and deployment verification items. Auditors should treat the pinned source and deployed bytecode/account state as authoritative for findings.

Revision since the prior audit package

Previous pinned commit: 602b43d76297ca3daa2cffffc76da6508eda52dc2

Current pinned commit: d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd

Change	Audit significance	Evidence / status
SBF stack fix - CancelTournament account boxing	The cancellation account context is now boxed, reducing stack pressure below Solana's 4,096-byte frame limit. This is a build-enablement change, but the larger account graph still requires constraint review.	Source diff / engineering-reported blocker resolution.
SBF stack fix - nimzovich_engine desktop transposition tables	Desktop-only stack-heavy constructors are gated away from Solana targets so the no_std move-validation subset can compile for SBF without pulling desktop search allocations into the BPF/SBF frame.	Engineering-reported blocker resolution; auditors should inspect cfg coverage and ensure no host-only search code enters the deployable feature graph.
On-chain move validation enabled by default	`Cargo.toml` now defaults to `cranks`, `move-validation`, and `devnet`; production expands to `move-validation` + `mainnet`. `record_move` derives the next board from the stored state.	Source-verified in current Cargo/lib/apply files.

Change	Audit significance	Evidence / status
ProgramConfig PDA added	Introduces `[b"config"]` state containing six authority keys, eight economic/timing parameters, bump, and 64 bytes reserved space; `initialize_config` and `update_config` are exposed as program instructions.	Source-verified in current commit. See the configuration section below.
Devnet buffer extended and program upgraded	The deployed executable has additional account capacity and a post-commit devnet deployment, which affects deployment reproducibility and bytecode/account-state verification.	Engineering-reported: buffer +100,000 bytes; deployed slot 497638332; exact deployed ELF/hash should be supplied to auditors.
Repository state	The audit baseline now includes commits `602b43d76` and `d7063b6fc` on `origin/main`, plus `.quarantine/` ignore changes.	Engineering-reported; verify remote HEAD and deployment artifact at audit kickoff.

IMPORTANT CONFIGURATION CAVEAT

The current `d7063b6fc` commit clearly adds `ProgramConfig` storage and update instructions, but the patch itself does **not** show broad migration of existing consumers away from `constants.rs`. For example, `create\_game` still imports and enforces `MAX\_WAGER\_AMOUNT` / `MIN\_WAGER\_LAMPORTS` from `constants.rs`, and settlement/dispute/crank paths still import constant values. Therefore the security property "fees/limits/authorities are instantly mutable" should be treated as **partially implemented pending call-site verification**, not assumed from the existence of the config account alone.

Executive summary

XFChess is a Solana Anchor program for wagered 1v1 chess, tournament registration and prize custody, player profiles/ELO, social relationships, session-key authorization, dispute resolution, and MagicBlock Ephemeral Rollup (ER) delegated game execution. The architecture intentionally splits live chess execution from financial settlement: the delegated Game PDA carries authoritative chess state, while SOL/USDC escrow, treasury, profiles, ELO, and final payout remain base-layer concerns.

The strongest security boundary is the on-chain move validator. With the default `move-validation` feature, `record\_move` re-derives the next board using the on-chain `nimzovich\_engine` no-std subset and rejects a caller-supplied `next\_board` that differs from the derived state. The full alpha-beta search is not part of the on-chain trust boundary; only legality/state transition is.

The second major boundary is delegation ownership. While a Game is delegated, the PDA is owned by the MagicBlock delegation program. Base-layer instructions that require `Account` therefore fail Anchor owner validation. Terminal instructions record a result on the delegated state, then `undelegate\_game` commits and returns ownership, after which `finalize\_game` performs value movement and profile/ELO settlement.

DEPLOYMENT-CRITICAL FINDING

At the pinned commit, `production = ["move-validation", "mainnet"]`; the mainnet program ID is a placeholder and the mainnet authority constants are zeroed, with compile-time guards that intentionally refuse a mainnet build until they are replaced. The pinned Anchor configuration is devnet/localnet, so this package should be understood as a devnet-targeted audit specification unless a separate deployment commit changes those guards.

Scope and evidence basis

Item	Pinned value
Repository	https://github.com/trilltino/XFChess
Source commit	d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd
Program directory	programs/xfchess-game
Program ID	8tevgspityTTG45KvvRtWV4GZ2kuGDBYWMXouFGquyDU
Anchor cluster in Anchor.toml	devnet
Anchor / Solana stack	Anchor 1.1.2 / Solana 3.0.0
Ephemeral Rollups SDK	0.16.2
Magic program API	0.13.11
Move engine	nimzovich_engine, no_std subset behind move-validation
Primary language	Rust / Anchor 2021 edition

Primary source evidence: [lib.rs](<https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/lib.rs>), [Cargo.toml](<https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/Cargo.toml>), [constants.rs](<https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/constants.rs>), [state module root](<https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/>). Additional architecture evidence is in [MAGICBLOCK.md](<https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/MAGICBLOCK.md>) and [docs/THREAT_MODEL.md](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/docs/THREAT_MODEL.md).

Out-of-scope / external systems

- Wallet private-key security, user device compromise, Phantom/Solflare browser security, and session-key private-key custody outside the program.
- MagicBlock delegation-program correctness beyond the assumptions explicitly called out here; auditors should independently review the pinned external program/SDK behavior relevant to ownership, commit, rollback, and timeout instructions.
- Backend correctness except where it changes on-chain trust assumptions, transaction routing, or the liveness/settlement boundary.
- The full alpha-beta chess search and local client UX; those are not part of the wager settlement security boundary.
- Off-chain FIDE pairing/social/chat/anti-cheat systems except where they supply inputs consumed by on-chain authorization or state transitions.

Definitions and trust model

Term	Meaning / trust position
Canonical state	Solana account state after a committed/undelegated Game; for active delegated games, the delegated Game account state is the active execution state subject to ER persistence assumptions.
ER	MagicBlock Ephemeral Rollup. Low-latency execution environment for delegated Game state.
Base layer	Solana execution/settlement path used for escrow, treasury, ELO, profiles, tournament funding and payouts.
Session key	A delegated signing key whose authority is bounded on-chain by session account, expiry, per-wager cap, cumulative spending cap, and operation flags.
VPS authority	A fixed privileged program key used for tournament administration and ER/time-check operations; it is not a generic owner key.
Host treasury	Tournament-specific <code>'host_treasury'</code> configured at initialization; the program later constrains mutable payout/funding paths against the stored value.
Dispute authority	Fixed key authorized to resolve disputes and attest certain recovery facts.
Link authority	Fixed key authorized to seed/link external rating metadata.
Upgrade authority	Solana deployment-level upgrade authority; not encoded as an ordinary instruction signer in this program. It can replace executable code if the deployed program remains upgradeable.

Privileged roles and authorities matrix

Role	Source / identifier	Capabilities	Security significance	Auditor verification
Program upgrade authority	Deployment metadata; not determinable from repository source alone	Can replace deployed program code while upgradeable	Critical; full program control	Query ProgramData on the target cluster and verify expected multisig/lock status.
ProgramConfig authority	<code>'ProgramConfig.authority' at '[b"config"]'</code>	Can rotate treasury/VPS/dispute/link/KYC keys and mutate configured wager/fee/dispute/crank parameters via <code>'update_config'</code>	Critical governance/financial privilege; can be changed instantly at state level	Verify PDA, current authority, key custody, rotation semantics, and every runtime consumer of each field.
VPS authority	Configured <code>'ProgramConfig.vps_authority'</code> ; compiled devnet constant remains in <code>'constants.rs'</code> until consumer migration is complete	Tournament initialization/start/shards/escrow administration and crank-linked operational paths as constrained by each account context	High privileged hot/admin role	Verify which source is authoritative at each call site and ensure config rotation actually takes effect.

Role	Source / identifier	Capabilities	Security significance	Auditor verification
Treasury authority	Configured `ProgramConfig.treasury_authority`; compiled devnet constant remains a legacy source until consumers migrate	Withdraws country treasury vault funds through `withdraw_treasury` constraints and receives treasury-directed value	High financial privilege	Verify configured key is used on every treasury constraint and payout path.
Dispute authority	Configured `ProgramConfig.dispute_authority`; compiled devnet constant remains a legacy source until consumers migrate	Resolve disputes and participate in stuck-delegation recovery attestation	High financial/claims privilege	Review key custody, quorum, audit trail, recovery procedures, and config migration completeness.
Link authority	Configured `ProgramConfig.link_authority`; compiled devnet constant remains a legacy source until consumers migrate	Authorize external ELO linkage/ratings	Integrity / reputation privilege	Verify source validation, signature semantics, and dynamic-config effectivity.
KYC / profile verification authority	Configured `ProgramConfig.kyc_authority`; compiled devnet constant remains a legacy source until consumers migrate	Verify player profile state where required	Identity/reputation privilege	Confirm governance around verification, privacy, and dynamic-config effectivity.
Game players	Game.white / Game.black	Move, resign, draw offer/accept, dispute as allowed by contexts	User-controlled state transitions only	Check every instruction binds signer to stored player identity.
Session key	Stored SessionDelegation / PlayerSession / GlobalSessionDelegation / TournamentSessionDelegation	Bounded operations without repeated wallet prompts	Potentially hot; bounded by on-chain caps	Verify no session path bypasses owner/session binding or caps.
MagicBlock delegation program	External SDK/program IDs	Owns delegated Game; executes ER path and commit/undelegate	External trust dependency	Pin exact program IDs/version and test owner transitions / rollback semantics.
Backend relayer	Off-chain service	Builds/routes transactions; should never be custody source	Availability/routing trust only	Confirm backend never signs user funds and cannot bypass on-chain constraints.

Build, cluster and deployment controls

Default features: cranks + move-validation + devnet
Production features: move-validation + mainnet
Mutually exclusive env features: localnet | devnet | mainnet
Mainnet guard: real program ID + non-zero authority keys required before build

CURRENT BUILD BASELINE

`programs/xfchess-game/Cargo.toml` currently declares `default = ["cranks", "move-validation", "devnet"]`; `production = ["move-validation", "mainnet"]`. This makes on-chain move validation part of the normal program feature set and ensures production cannot build without it.

Engineering-reported SBF remediation: the previous `CancelTournament` account context exceeded the 4,096-byte Solana stack frame and was remediated by boxing the large Anchor accounts. The chess-engine desktop transposition-table constructors were also gated to `std` + non-Solana targets so the SBF build does not inherit the approximately 129 KB desktop stack allocations. These claims should be reproduced by the auditor with a clean `cargo build-sbf` / artifact hash comparison.

Engineering-reported devnet deployment: program data was extended by 100,000 bytes and the program was upgraded on devnet at slot 497638332 for program 8tevgspityTTG45KvvRtWV4GZ2kuGDBYWMXouFGquyDU. The audit handoff should include the post-upgrade program-data account, deployed ELF hash, upgrade transaction signature, and the exact SBF build command/features used to produce that bytecode.

The pinned `Anchor.toml` config points the provider to devnet and maps both devnet and localnet to the provided program ID. This is consistent with the source-side `declare\_id!` gating. A production audit should pin the exact production feature set, compiled artifact hash, deployment transaction/signature, ProgramData address, upgrade authority, and external MagicBlock IDs actually used in production.

Dependency and trust map

Dependency	Pinned version / role	Security boundary
Anchor	1.1.2	Accounts constraints, serialization, instruction dispatch, owner checks, CPI framework.
Solana	3.0.0	Runtime, system transfers, account ownership, time/clock, upgradeable-loader deployment semantics.
ephemeral-rollups-sdk	0.16.2	Delegate/commit/undelegate ownership flow; MagicBlock account plumbing.
magicblock-magic-program-api	0.13.11	Scheduled task / crank API when `cranks` enabled.
SPL Token / Associated Token	Anchor SPL 1.1.2 stack	USDC prize mint/token-account handling.
nimzovich_engine	workspace dependency, no_std for program	Chess legality/state transition; must be treated as consensus-critical code for move validation.
Backend / Magic Router	off-chain; routing only	Determines endpoint, not game legality or payout values; liveness/availability dependency.

Dependency	Pinned version / role	Security boundary
Iroh/Braid etc.	off-chain	Realtime UX/sync; not source of truth for wagered state.

PDA and account architecture

The table below distinguishes seed prefixes from the full tuple when the exact instruction context is distributed across multiple files. Auditors should derive the final canonical address from the concrete ``#[account(seeds=...)]`` declaration at the pinned commit rather than from documentation prose alone.

Account	Seed prefix / tuple	Struct	Owner	Purpose
Game	game	u64 game_id LE bytes	Game	program-owned; temporarily delegation-program-owned while ER delegated
Program configuration	config	literal b"config"	ProgramConfig	program
Wager escrow	escrow	u64 game_id LE bytes	System / zero-data PDA expected for SOL custody	System-owned in SOL transfer path
Player profile	profile	player authority pubkey	PlayerProfile	program
Username record	username	username bytes (instruction-defined)	UsernameRecord	program
Lichess username record	lichess_username	linked Lichess username (instruction-defined)	LichessUsernameRecord	program
Friendship	friendship	requester + addressee (instruction-defined)	Friendship	program
Per-game session	session_delegation	game_id + player	SessionDelegation	program
Tournament	tournament	u64 tournament_id LE bytes	Tournament	program
Tournament players shard	tourney_players	shard_id + tournament_id	TournamentPlayersShard	program
Tournament escrow	t_escrow	u64 tournament_id LE bytes	TournamentEscrow (zero-data)	program
Tournament match	t_match	tournament_id + match_index (instruction-defined ordering)	TournamentMatch	program
USDC prize authority	t_usdc_prize	u64 tournament_id LE bytes	PDA authority + ATA	program authority / SPL Token ATA

Account	Seed prefix / tuple	Struct	Owner	Purpose
Treasury vault	treasury_vault	country context; seed prefix is treasury_vault	TreasuryVault / lamports	program
Player session	player_session	player (per state constant)	PlayerSession	program
Tournament session	tournament_session	player + tournament (state constant + instruction constraints)	TournamentSessionDelegation	program
Global session	global_session	player	GlobalSessionDelegation	program
Dispute	dispute	game_id / instruction-defined	DisputeRecord	program

Core state layouts

Game

Field	Type	Security / semantic role
game_id	u64	PDA discriminator input; uniqueness is address-level.
white / black	Pubkey	Participant principals.
status	GameStatus	Lifecycle state; combined with is_delegated to derive phase.
last_move_timestamp	i64	Legacy/activity timestamp for clock/monitoring.
fees_advanced	u64	Accumulates advanced flat costs reimbursed from escrow at settlement.
fee_payer	Pubkey	Must match across create/join/settlement/delegation paths.
result	GameResult	None / Winner(pubkey) / Draw.
board_state	[u8;68]	Compact authoritative board encoding.
move_count	u16	Number of recorded moves.
halfmove_clock	u16	50-move-rule counter.
turn	u16	Odd/even turn determines expected white/black signer.
created_at / updated_at	i64	Lifecycle/activity timestamps.
wager_amount	u64	Per-player base stake amount in SOL/token branch.
wager_token	Option	Optional non-SOL wager mint path.

Field	Type	Security / semantic role
game_type	GameType	Currently PvP.
match_type	MatchType	Free or Rated.
country_fee	u64	Country-fee component used at rated settlement.
base_time_seconds / increment_seconds	u64 / u16	Chess clock configuration.
bump	u8	Canonical PDA bump.
is_delegated	bool	Program-side mirror of delegation lifecycle.
tournament_id	Option	Optional tournament linkage.
nonce	u64	Replay/causal ordering.
draw_offered_by	Option	Pending draw offer state.

Tournament and shard state

`Tournament` stores authority, entry/platform fees, player counts, bracket/Swiss configuration, up to ten placements, 10,000-bp prize shares, SOL and optional USDC prize pools, host treasury, payout type, vesting parameters, clock controls, and round-report bitmap state. `TournamentPlayersShard` stores up to 64 players, ELOs, and Swiss standings per shard; shard capacity is fixed at 64, enabling 64/128/256-player layouts.

Profile / sessions / governance

State	Key controls
PlayerProfile	Wallet authority, country, wins/losses/draws, Glicko-like rating fields, wager totals, username, external ELO linkage, verification flag.
PlayerSession	24h default duration, 0.5 SOL default cumulative spending limit, 10 SOL default per-wager max, operation booleans, cumulative spend.
GlobalSessionDelegation	30-day default, 200-game default, 5 SOL cumulative default, 1 SOL per-wager default; invalid if disabled/expired/no games remaining.
TournamentSessionDelegation	48h default, 1 SOL cumulative default, 0.25 SOL per-wager default; bounded by tournament/player context.
DisputeRecord	Game ID, challenger, reason, evidence hash, status, resolver, resolution, timestamps, bond.
TreasuryVault	Country, treasury authority, cumulative lamports collected, bump.
Friendship	Requester, addressee, Pending/Accepted/Blocked, timestamps, bump.

ProgramConfig architecture and governance surface

Field	Type	Meaning	Audit question
authority	Pubkey	Super-admin that can update the config PDA.	Is this key governed by a multisig/timelock and can it rotate itself safely?
treasury_authority	Pubkey	Configured treasury/fee recipient authority.	Do runtime treasury constraints actually read this field, or do they still use compiled constants?
vps_authority	Pubkey	Automated crank / time-check authority.	Does every crank/admin path use the config field consistently?
dispute_authority	Pubkey	Dispute arbiter key.	Does dispute resolution read the config field on every path, including recovery/stale paths?
link_authority	Pubkey	External ELO linkage signer.	Is external ELO authorization sourced from config everywhere?
kyc_authority	Pubkey	KYC/profile verification authority.	Is verification authorization sourced from config everywhere?
max_wager_amount	u64	Maximum allowed SOL wager amount.	Trace create/join/session/global/tournament game paths; no stale constant path may bypass the configured ceiling.
min_wager_lamports	u64	Minimum nonzero wager.	Verify every wager entry point uses the same configured floor.
max_platform_fee_lamports	u64	Maximum platform fee.	Verify fee validation and settlement use the same configurable ceiling.
er_session_fee_lamports	u64	MagicBlock/ER session fee.	Verify authorization and billing paths consume this field if advertised as dynamic.
dispute_bond_lamports	u64	Bond required to open a dispute.	Verify dispute creation reads config rather than the compiled constant.
dispute_ttl_secs	i64	Pending-dispute expiration.	Verify dispute creation/expiry uses this dynamic value consistently.
crank_max_slot_delay	u64	Maximum tolerated slot lag for crank checks.	Verify crank scheduling/execution uses config consistently.
crank_max_seconds_early	i64	Allowed early trigger jitter.	Verify crank acceptance checks use config consistently.
bump	u8	Canonical config PDA bump.	Must match PDA derivation; never caller supplied.

Field	Type	Meaning	Audit question
reserved	[u8;64]	Reserved zero-initialized future migration space.	Future additions must preserve account layout and discriminator invariants.

Initialization: `initialize_config` creates the PDA with seeds `[b"config"]`, sets the initializer as `authority`, and uses provided values or current compiled constants as defaults. **Updates:** `update_config` checks `config.authority == authority.key()` and then applies each optional field. **Critical audit point:** configuration storage is only economically meaningful if every intended consumer has migrated from `constants.rs` to reading this PDA. The current commit should therefore be audited as a governance/state feature plus a migration-completeness review.

DYNAMIC-CONFIG GOVERNANCE TRADE-OFF

A live configuration PDA converts what used to be deployment-time changes into privileged state changes. This lowers operational deployment cost but increases governance blast radius: a compromised config authority can change limits, fee ceilings, dispute timing, and role keys without a code upgrade. Auditors should model this key as a high-severity governance principal and require an explicit rotation/incident-response procedure.

Game state machine

```

WaitingBase: create_game -> WaitingForOpponent
WaitingBase -> ActiveBase: join_game
ActiveBase -> ActiveER: delegate_game
ActiveER -> ActiveER: record_move / global_record_move
ActiveER -> FinishedER: checkmate / resignation / timeout / draw agreement
FinishedER -> FinishedBase: commit + undelegate
FinishedBase -> Settled: finalize_game
WaitingBase -> Cancelled: cancel_game
ActiveBase -> Cancelled: cancel_game
Disputed: dispute_game -> resolve_dispute / stale path
Wiped after force recovery -> recover_stuck_delegation -> escrow-only recovery

```

The source-level `GamePhase` explicitly maps `(status,is_delegated)` to `WaitingBase`, `ActiveBase`, `ActiveEr`, `FinishedEr`, `FinishedBase`, `Settled`, `Cancelled`, or `Disputed`. Impossible combinations return `InvalidLifecycleTransition`. This should be treated as a core invariant in every audit test.

Game instruction matrix

Instruction	Authority / signer	Primary accounts	State effects	Value effects
<code>create_game</code>	player + fee_payer	Game PDA; escrow PDA	Creates <code>WaitingForOpponent</code> ; initializes board/timers/fees	If <code>wager > 0</code> , transfers <code>wager_amount</code> from player to escrow.
<code>join_game</code>	player + fee_payer	Game; player/white profiles; escrow	<code>Waiting</code> → <code>Active</code> ; sets black; updates activity	If SOL wager, transfers matching stake to escrow.

Instruction	Authority / signer	Primary accounts	State effects	Value effects
record_move	session key	Game; SessionDelegation	Validates turn, nonce, board transition; increments move/turn; may finish	No escrow movement; records advanced fee counter.
global_record_move	global session key	Game; GlobalSessionDelegation	Same authoritative move transition using global session	No escrow movement; records advanced fee counter.
resign	participant	Game	Active → Finished; winner is opponent	None; settlement later.
offer_draw	participant	Game	Stores draw_offered_by	None.
accept_draw	other participant	Game	Active → Finished Draw	None; settlement later.
claim_timeout	permissionless signer	Game	Active → Finished winner or Cancelled if no moves	None; settlement later.
cancel_game	participant / constrained player path	Game; escrow; player authorities	Waiting/eligible Active → Cancelled	Refunds escrow stake amounts in base-layer handler.
finalize_game	fee payer + player authorities	Game; profiles; escrow; treasury vault	Finished/Cancelled → Settled; closes Game	Pays transaction/reimbursement/treasury/ELO fees and terminal winner/draw or cancellation refund.
withdraw_expired_wager	creator	Game/escrow under expiry guards	Recovery-specific terminal state	Withdraws eligible expired wager subject to expiry/ownership constraints.

Move validation and anti-replay

`record\_move` accepts `(game\_id, move\_uci[5], next\_board[68], nonce, optional signature bytes, optional parent\_nonce)`. The current pinned implementation does not use the optional signature blob as the authority mechanism; the actual authorization is the session account constraint tying `session\_key == signer` and `session\_delegation.player` to the game participant. The signature parameter is therefore not a source of trust and must not be described to integrators as an independently verified cryptographic attestation.

The apply path checks Active status, expected player from turn parity, parent nonce when provided, and exact next nonce. With move validation enabled it parses the stored compact board, calls `validate\_and\_apply`, compares the derived compact board against the client-proposed board, updates the halfmove clock, and detects checkmate/stalemate/insufficient-material/50-move draws. Only after these checks is the submitted board committed to Game state.

Terminal game semantics

Terminal trigger	Result	Immediate state	Later financial effect
Checkmate	Winner(moving player)	Finished; delegated flag preserved until undelegation	Winner paid at finalization.
Resignation	Winner(opponent)	Finished	Winner paid at finalization.
Draw agreement	Draw	Finished	Remaining pot split using draw split helper.
Stalemate	Draw	Finished	Draw split at settlement.
Insufficient material	Draw	Finished	Draw split at settlement.
50-move rule	Draw	Finished	Draw split at settlement.
Timeout after at least one move	Winner(other side)	Finished	Winner paid at settlement.
Timeout before any move	None	Cancelled	Neutral refund path.
Cancellation	None	Cancelled	Stake refund path, subject to exact cancellation guard.

Game escrow and SOL lifecycle

```

Creator -> escrow PDA: wager_amount
Opponent -> escrow PDA: wager_amount

At finalization (rated example):
escrow -> fee_payer: limited tx-fee reimbursement
escrow -> treasury_vault: advanced platform/operational reimbursement
escrow -> treasury_vault: country fee (rated path)
escrow -> white/black: ELO fee reimbursement split
escrow -> winner OR draw recipients: remaining pot

Cancelled: escrow -> white stake; escrow -> black stake (if joined)

```

The exact remaining-pot arithmetic is centralized in the settlement module. The source explicitly limits each payment by the remaining pot, uses checked/saturating arithmetic for intermediate values, and requires player destinations to remain rent-exempt after payout. Auditors should verify both the conservation equation and the rent-exemption precondition for every wager type.

Settlement accounting equations

Quantity	Definition / expected invariant
Gross escrow funded	Sum of creator and joiner deposits for a two-player SOL game, normally $2 * \text{wager_amount}$.
Available pot	<code>`escrow::pot(wager_amount)`</code> as defined by the canonical escrow helper; auditors should inspect helper implementation for exact treatment of rent/minimum balance.

Quantity	Definition / expected invariant
Net distributable	Gross pot less transaction-fee reimbursement, fee reimbursement, country fee, and rated ELO fee split, bounded below by zero.
Winner settlement	Winner receives the remaining distributable amount after all preceding deductions.
Draw settlement	Remaining distributable amount is split by the canonical draw-split helper; both recipients must remain rent-exempt after transfer.
Conservation invariant	Sum(all outbound escrow transfers) <= initial spendable escrow pot; no branch may create lamports or pay from an unrelated account.
Profile invariant	Finalization changes profiles only after result/status validation and only in the base settlement path.

Tournament state machine

```

Registration
| initialize_tournament + initialize_shards + initialize_tournament_escrow
| register_player / leave_tournament
v
Active
| single elimination: initialize_match -> record_match_result -> advance_winner
| Swiss: record_swiss_result -> advance_round -> ... -> complete_swiss_tournament
v
Completed
| distribute_tournament_prizes / claim_tournament_prize
v
Closed

Cancellation may transition Registration/eligible states -> Cancelled, followed by fund/refund/close guards.

```

Tournament initialization and capacity

Valid `max\_players` values are exactly 2, 4, 8, 16, 32, 64, 128, or 256. Shard capacity is fixed at 64. Small tournaments use one shard, medium tournaments use two, and 256-player tournaments use four. Tournament initialization enforces name length <= 64 bytes/characters at the source check, `elo\_min <= elo\_max`, `min\_players <= max\_players`, and prize share sum <= 10,000 basis points. Winner-takes-all overrides configured shares with 10000 bp to first place.

Tournament registration and entry economics

Registration operates against Tournament status, capacity, duplicate-registration checks, and the configured ELO range. The tournament state keeps both a prize pool and platform-fee bookkeeping, so auditors should distinguish player entry deposits from host/platform fee balances. Leaving/cancellation must be audited as a conservation problem: every refundable deposit must map to an entitled participant, and no participant should be able to register twice or withdraw twice.

Tournament prize lifecycle

Phase	SOL / USDC action	Audit invariant
Initialize	Store mint/payout configuration and host treasury; create escrow/control PDAs.	Configuration cannot be mutated by arbitrary callers.
Fund SOL	Transfer SOL into tournament prize custody.	Recorded funding cannot exceed actual custody shortfall; repeated funding must be additive or explicitly guarded.
Fund USDC	Transfer from funder token account into prize ATA/control account.	Mint must match configured USDC prize mint; Token CPI failures abort.
Distribute	Compute placement amounts from prize shares / payout type; store claim/distribution state.	Total allocation $\leq$ funded pool.
Claim	Winner/placement claims once.	Double claim rejected; non-winners rejected; aggregate claimed $\leq$ funded.
Close	Close only once prizes are no longer outstanding.	Outstanding funded places block closure.

Swiss architecture

Swiss state is compact and deliberately separates registration identity storage from per-player standings. Each shard holds players, player ELOs, and `SwissStanding` records containing score, Buchholz, Sonneborn-Berger, and color balance. Round/board result writes are expected to be protected by the round-board bitmap and player identity checks. `advance\_round` is designed to be permissionless once player-signature evidence/result persistence satisfies the guards; this is an important liveness property because it reduces dependence on a single tournament administrator.

Single-elimination architecture

`TournamentMatch` contains optional white/black/winner, linked Game PDA/game ID, match status, next-match index/slot, and start/completion timestamps. `initialize\_match` creates the match, `record\_match\_result` records winner/loser, and `advance\_winner` moves the winner into the configured downstream bracket slot. Auditors should treat match-link integrity as a graph invariant: a winner may advance only to the target encoded by the canonical source match, and a match must not complete twice.

Tournament session-key architecture

Tournament sessions bound a session key to both a tournament and a player and use cumulative spending + maximum wager caps. Tournament session creation/join paths should be considered a financial authorization surface because the session signer is not the wallet owner. The primary invariant is that every PDA-funded game still carries the originating player as its economic principal and cannot be created against a different profile, tournament, or fee payer.

Global persistent session architecture

The global session PDA is keyed by player and contains a session signer, expiry, cumulative spending cap, maximum wager, remaining-game quota, enable bit, and bump. At the pinned commit the default duration is 30 days, the default game quota is 200, cumulative spending limit is 5 SOL, and per-wager cap is 1 SOL.

`global\_create\_game` additionally checks the session PDA has enough lamports for the Game account rent, wager funding, and preservation of its own rent floor before it debits the PDA.

Delegation lifecycle and MagicBlock trust boundary

The delegation path serializes the Game state with `is\_delegated=true` before invoking the MagicBlock CPI because the CPI changes the account owner. The default delegation configuration uses a 30-second commit frequency and leaves validator selection to MagicBlock. This makes MagicBlock a trusted execution/persistence dependency for live games but not a trusted source of chess semantics: legality remains in the same on-chain Rust transition logic.

```
Solana base
  create_game / join_game
    |
    v
  delegate_game
    | owner -> delegation program
    v
MagicBlock ER / Router
  record_move / resign / timeout / draw
    |
    v
  commit + undelegate
    | owner -> XFChess program
    v
Solana base
  finalize_game -> payout / ELO / profiles
```

ER hot-path write boundary

Instruction class	Allowed while Game delegated?	Reason
record_move / global_record_move	Yes	Writes only Game PDA plus reads session delegation; account owner context is ER/delegated.
resign / offer_draw / accept_draw / claim_timeout	Yes	Terminal/game-state-only writes; no escrow/profile/token account movement.
delegate_game	Base only before delegation	Establishes delegation.
undelegate_game	ER/base boundary	Returns Game to base ownership after commit.
finalize_game	No	Financial settlement and profile mutation require base ownership.
cancel_game / tournament / treasury	No for delegated game	Base-layer accounts require program ownership; delegated Game owner check blocks invalid use.

ER unavailability and forced recovery

DATA-LOSS RECOVERY SEMANTICS

The repository's MagicBlock integration explicitly documents the forced-undelegation path as a wipe, not a rollback to a good snapshot. `request_force_undelegate` starts an external timeout; `force_undelegate_after_timeout` can return the Game PDA with zero-length/absent serialized game state. `recover_stuck_delegation` then handles the resulting empty-account dead end using dispute authority attestation of the players and splits recoverable escrow 50/50. It intentionally does not update ELO or profiles.

This is a core trade-off and should be in the audit report: ER persistence failure can destroy the rich Game state, but the design aims to prevent permanent custody loss by moving to a neutral escrow-refund recovery. The audit question is therefore not whether recovery preserves game state - it does not by design - but whether an attacker can trigger recovery against a live game, forge player identities, drain escrow more than once, or convert a non-stuck game into a recoverable state.

Dispute model

A `DisputeRecord` stores game ID, challenger, up to 200-character reason, 32-byte evidence hash, Pending/Resolved/Dismissed status, optional resolver, resolution text, timestamps, 7-day expiry, bond amount, and bump. The intended financial model is that a disputed game cannot be silently settled in a contradictory way while the dispute remains active. The stale-dispute path is specifically designed to avoid indefinite escrow lock if the authority does not resolve the dispute.

Profiles, usernames, external ELO

Profile state is more than presentation metadata: it feeds player authorization, ELO settlement, annual winnings counters, verified status, and external ELO seeding. Username validation enforces 3-20 characters, ASCII alphanumeric plus underscore/hyphen, and a reserved-prefix list. The separate `UsernameRecord` makes uniqueness an account-level invariant rather than relying only on a mutable string in `PlayerProfile`. `LichessUsernameRecord` similarly creates an on-chain uniqueness anchor for a linked Lichess username.

The audit should verify that `link_external_elo` authenticates the Link Authority, binds the linked username to the caller wallet, and cannot overwrite another wallet's uniqueness record. The pinned repository includes a regression test specifically covering one-Lichess-username-per-wallet uniqueness.

ELO and financial profile settlement

Rated games update the time-control-specific rating field selected by `bucket_for_time_control`. The pinned settlement code calculates the new pair of ratings, updates last-played timestamps, win streak/best streak fields, annual-winnings country counters, and wager/game statistics according to result. This means ELO/profile integrity is coupled to settlement finality: if settlement can execute twice, ratings/statistics can double-apply even if the escrow itself is already empty. The `Settled` transition plus Game account closure is therefore a security boundary, not merely bookkeeping.

Treasury and fee architecture

The `TreasuryVault` is country-scoped state containing authority and cumulative collected lamports. Game settlement can pay the treasury vault for advanced platform/operational costs and country fees;

``withdraw_treasury`` requires the stored treasury authority. Tournament state also stores a ``host_treasury`` Pubkey, which is distinct from the fixed global treasury authority. Auditors should model these as two different trust domains: protocol treasury withdrawal vs tournament-specific payout destination.

Time, crank, and timeout semantics

The chess clock is represented on Game by base time, increment, and activity timestamps. ``claim_timeout`` checks elapsed inactivity against the source timeout helper. When the cranks feature is enabled, MagicBlock scheduled tasks can run the timeout logic on the ER. The repository documents scheduling/cancellation as best-effort operational calls; failure to schedule is a liveness defect but should not itself create a financial double-spend.

The key boundary is strict time comparison: session validity uses ``now < expires_at``, while timeout requires elapsed time to be greater than the inactivity window. Auditors should test boundary timestamps exactly, including timestamp arithmetic near i64 limits and negative/clock skew scenarios.

Error surface and rejection taxonomy

The program centralizes a large error enum covering authorization, lifecycle, move validity, session expiry, spending limits, ELO bounds, tournament state, funding, prize claims, timeouts, arithmetic overflow, usernames, and recovery. This is useful for auditors because it makes negative-path coverage enumerable. The high-value rejection families are: wrong signer; wrong PDA/authority; wrong lifecycle; delegated/base mismatch; replay/nonce mismatch; invalid board; insufficient escrow; duplicate claim/registration; expired session; and invalid tournament status.

Security invariants for an external auditor

ID	Invariant	Audit test expectation
I-01	PDA integrity	Every privileged state account must be constrained by canonical program seeds and/or stored authority relationships; no caller-chosen state account may substitute for a canonical PDA.
I-02	Game phase coherence	<code>`Game::phase()`</code> must reject impossible <code>`(status, is_delegated)`</code> pairs; in particular a waiting game cannot be delegated and settlement phases require undelegated state.
I-03	Terminal result before settlement	<code>`finalize_game`</code> only accepts <code>`Finished`</code> or <code>`Cancelled`</code> ; a Finished game must have a non-None result.
I-04	Delegated settlement exclusion	A delegated Game cannot be passed through base-layer value-moving settlement as a normal Anchor <code>`Account`</code> ; explicit undelegation is required.
I-05	Move turn integrity	The signer/session principal must correspond to the player whose turn is expected from the turn counter.
I-06	Move causality	Move nonce must equal prior nonce + 1; optional parent nonce must equal on-chain current nonce before increment.

ID	Invariant	Audit test expectation
I-07	Authoritative board transition	With `move-validation` enabled, the on-chain engine derives the next board from stored board + UCI move and rejects a mismatching client `next_board`.
I-08	Terminal chess detection	Checkmate creates Winner; stalemate/insufficient material create Draw; 50-move rule is detected at halfmove_clock >= 100.
I-09	Session expiry	Per-game/global/tournament sessions require enabled state and current time strictly before expiration, where the session type defines additional quotas.
I-10	Session budget	A session-funded wager must satisfy both per-wager cap and cumulative spending cap using checked arithmetic.
I-11	Wager bound	Create/join paths enforce `wager_amount == 0    wager_amount >= 1_000_000` and `wager_amount <= 10 SOL`.
I-12	Escrow conservation	Value paid from game escrow must be sourced from the canonical escrow PDA; each settlement branch must not pay more than the available pot/remaining value.
I-13	Refund neutrality for cancellation	Cancellation of a two-player SOL wager returns the defined player stakes rather than routing a win to one side.
I-14	Settlement once	Game settlement transitions to Settled and the Game account is closed; the canonical settlement path is the only place that pays the terminal prize and updates profiles.
I-15	ELO coupling	Rated/non-free finalization performs the configured rating update for the correct time-control bucket and updates associated player statistics.
I-16	Draw agreement authenticity	Only a participant can offer; only the other participant can accept; an offer alone does not finish the game.
I-17	Timeout authenticity	Timeout requires Active status and elapsed inactivity greater than the computed timeout window; no-move timeout resolves to cancellation/neutral state.
I-18	Dispute bond/TTL	A dispute is bonded and has an explicit expiration; stale-dispute behavior must not mint value or produce a contradictory terminal game result.
I-19	Tournament capacity	Valid tournament max-player counts are 2/4/8/16/32/64/128/256; shards are selected to fit the fixed shard capacity.
I-20	Prize share conservation	Configured prize shares sum to at most 10,000 basis points; winner-takes-all forces a 10000-bp first-place allocation.

ID	Invariant	Audit test expectation
I-21	Tournament registration uniqueness	A wallet cannot register twice for the same tournament; registration is allowed only in Registration state and within capacity.
I-22	Tournament funding prerequisite	Guaranteed prize funding must be complete before registration opens/starts where the selected payout model requires it; funded pools cannot be replaced arbitrarily.
I-23	Prize claims never exceed funding	Aggregate claimed/distributed tournament prizes must be bounded by the funded prize pool(s), and each winner/placement claim is single-use.
I-24	Tournament completion gating	Close requires Completed/terminal status and rejects outstanding funded prize places; Swiss completion requires all rounds advanced.
I-25	Swiss replay protection	Round/board results are bitmap/slot protected so the same board cannot be recorded twice; pairwise player identity is checked.
I-26	Session-backed PDA funding	Global session create-game spends only from the session PDA after verifying player/session relationship, signer session key, spending budget, and rent viability.
I-27	ER write boundary	While delegated, ER hot-path writes are restricted to Game state; escrow, treasury, profiles, and player lamports remain base-layer concerns.
I-28	Mainnet build guard	Production/mainnet cannot build with placeholder program ID or zero authority keys; those compile-time guards must pass before a mainnet release exists.
I-29	Config PDA uniqueness and authority	`initialize_config` must create the single canonical `[b"config"]` PDA; `update_config` must require the stored `authority` signer and atomically apply only requested fields.
I-30	Config effectivity	A parameter advertised as dynamically configurable is not a security control until the consuming instructions read `ProgramConfig`; auditors should trace every configured field to its runtime call sites and reject stale hardcoded-constant paths.
I-31	Config authority rotation continuity	When `new_authority` is set, the new key becomes the sole future updater; the old key must lose update rights immediately and no alternate path may mutate the same config fields.

Threat model

Threat actor / failure	Desired security property	Primary controls
Malicious player	Cannot move for opponent or alter board illegally.	Turn parity, session-player binding, on-chain engine, next-board equality, nonce/parent nonce.

Threat actor / failure	Desired security property	Primary controls
Compromised session key	Cannot exceed granted game/tournament/global limits.	Expiry, enabled bit, max wager, cumulative spending, game count, player binding.
Malicious backend	Cannot steal or fabricate funds by changing transaction construction.	Backend does not own player keys; on-chain account constraints/PDA seeds decide authorization and destinations.
ER validator/RPC outage	Player funds are recoverable rather than permanently trapped.	Force-undeleagation timeout + recovery instruction; documented neutral escrow split.
Compromised dispute authority	Can materially affect disputed outcomes/recovery.	Fixed authority separation; bond/TTL; audit of governance key custody.
Compromised treasury authority	Can drain accumulated protocol treasury, but not arbitrary game escrow if settlement path is correct.	Vault-specific PDA + stored authority constraint.
Compromised upgrade authority	Can replace entire program logic and steal funds.	External deployment custody; multisig/timelock/immutability should be an explicit audit/deployment decision.
Malformed token mint/account	Cannot redirect prize tokens to arbitrary mint/account.	Configured mint, ATA/account constraints, SPL Token CPI.
Replay of move/result	Cannot apply same state transition twice.	Nonce + parent nonce + terminal status guards.
Double settlement	Cannot pay winner/ELO twice.	Finished → Settled transition and close semantics.
DoS through oversized state	Cannot force rent/compute beyond fixed bounds unexpectedly.	Fixed tournament capacities, shard capacities, max batch/session limits, bounded strings.

Known trade-offs and residual trust assumptions

- **MagicBlock persistence:** live game availability depends on ER/Routing. The recovery path is intentionally lossy for game state.
- **On-chain chess compute:** legality/state transition is consensus-critical but constrained by Solana compute/account limits; the program therefore uses a compact board representation and excludes full AI search.
- **Session keys:** improve UX but expand the authorization surface. A compromised session key is intentionally powerful within its configured budget and operation set.
- **Backend routing:** routing is off-chain, but the backend cannot make an invalid board or bypass settlement ownership checks. Availability failures remain possible.
- **Governance concentration:** dispute, treasury, Link, KYC, and VPS authorities are single fixed keys in the pinned devnet build. Operational security of these keys is a first-class trust assumption.
- **Upgradeable deployment:** unless the deployed ProgramData is made immutable, the upgrade authority is ultimately trusted with contract logic. This is outside normal instruction-level authorization and must be reported separately.
- **Client-proposed next board:** the client still supplies `next\_board`, but the program verifies it against the derived board. This is a bandwidth/serialization interface rather than a trust assumption when move validation is enabled.

Audit focus areas / suggested test matrix

Priority	Area	Concrete tests
P0	Upgrade/deployment	Verify deployed bytecode matches commit/build artifact; inspect ProgramData upgrade authority; verify mainnet feature/build guards; verify external MagicBlock program IDs.
P0	ProgramConfig migration	For every field in ProgramConfig, trace all current consumers; demonstrate that updating config changes the intended runtime behavior; attempt unauthorized update and authority rotation; verify stale constant paths do not remain security-relevant.
P0	Escrow conservation	Create/join/resign/draw/timeout/cancel/finalize under min/max/near-zero/near-max wagers; assert exact lamport deltas and no overpayment.
P0	Delegation boundary	Attempt finalize/cancel/treasury/profile writes while delegated; attempt delegate twice; undelegate wrong game/magic accounts; attempt forced recovery before/after timeout.
P0	Move validator	Illegal UCI, legal move with bad next board, wrong player, repeated nonce, skipped nonce, wrong parent nonce, checkmate, stalemate, insufficient material, 50-move draw, promotion/en-passant/castling.
P0	Session authorization	Wrong session signer; wrong player in session; expired; disabled; max-wager +1; cumulative cap +1; games_remaining=0; near-u64 overflow.
P0	Tournament prize accounting	Prize shares >10000; zero funded pool; partial funding; repeated funding; claim twice; non-winner claim; final claim; close with outstanding prizes; SOL and USDC branches.
P1	Disputes	Duplicate dispute, non-player dispute, expired claim, authority replacement attempt, double resolution, resolution after settlement, recovery against non-wiped Game.
P1	Swiss/bracket	Duplicate board result; wrong round/board; invalid player; double match completion; wrong next-match link; round advancement before all required results.
P1	Identity/ELO	Duplicate username; reserved prefix; cooldown; duplicate Lichess username; incorrect Link Authority; mismatched profile authority; rated settlement statistics.
P1	Time arithmetic	Exact expiry, timeout boundary, timestamp overflow/underflow, base-time zero, very large increments, crank schedule/cancel race.
P2	Social state	Friendship PDA collision, self-friend, stale status, block/accept ordering, close idempotence.

Static source review checklist

1. Confirm every `#[derive(Accounts)]` struct binds each writable account to the intended PDA seeds or stored authority constraint.
2. Confirm every CPI transfer from a PDA uses the exact canonical seeds/bump for that PDA and never a caller-supplied arbitrary seed tuple.
3. Confirm every terminal result instruction is state-only on ER and cannot receive escrow/token accounts through remaining accounts or unchecked account paths.
4. Confirm every settlement branch is impossible while delegated and cannot be reached against a forged/zero-data Game account.
5. Confirm all arithmetic that affects value uses checked operations or a bounded helper with an explicit conservation invariant.
6. Confirm token prize branches bind mint, token account owner/authority, and destination recipient consistently.
7. Confirm all session-funded operations debit the same session PDA whose budget is checked and whose player is bound to the economic principal.
8. Confirm `global_create_game` cannot escalate signer privileges via PDA signing/CPI account metas, using the dedicated regression test as a starting point.
9. Confirm the move engine crate/version/features are reproducibly pinned and that `production` cannot compile without `move-validation`.
10. Confirm deployed program ID, ProgramData upgrade authority, config keys, treasury destinations, and MagicBlock IDs from on-chain state, not solely repository constants.
11. Confirm every advertised ProgramConfig parameter has a live consumer; where a consumer still imports `constants.rs`, document the mismatch explicitly and treat the config field as non-authoritative until migrated.

Test-suite evidence and auditability

The current Cargo manifest adds a dedicated `config_tests` integration suite alongside the existing ER, settlement, tournament, session, ELO, dispute, and identity suites. The configuration test initializes the `[b"config"]` PDA, deserializes `ProgramConfig`, verifies authority and selected limit values, then performs an authorized update. The source does not itself assert compute-unit usage; any claim such as `<9k CU` or a particular lamport fee should therefore be treated as engineering-reported benchmark data and reproduced from a clean transaction trace.

Program-test does not reproduce live MagicBlock delegation, asynchronous commit, or undelegation behavior; auditors should pair these tests with a live devnet lifecycle test and verify the post-upgrade deployed artifact.

Recommended end-to-end audit scenarios

1. **Wagered ER game:** create → join → authorize session → delegate → 20 legal moves → checkmate → commit/undelegate → finalize → verify exact escrow, treasury, winner, profiles, ELO.
2. **Draw:** create → join → delegate → offer draw → accept by other player → undelegate → finalize → verify exact half/odd-lamport split helper behavior.
3. **Timeout:** use short base time → delegate → allow timeout crank → verify winner/terminal result → undelegate → finalize; repeat with zero moves and expect cancellation refund.
4. **Dispute:** finish game → dispute under supported window → resolve winner/draw → verify only authorized resolver can mutate outcome; test stale expiry.

5. **ER outage:** delegated Game → request force undelegation → wait required slot window → force undelegate → verify Game is wiped → recover_stuck_delegation → verify exactly 50/50 recoverable escrow and no ELO/profile mutation.
6. **Swiss tournament:** initialize 16/64/128/256 variants → shards → registrations at ELO bounds → record results with duplicate-board attempts → advance rounds → complete → fund/claim/close.
7. **Session abuse:** derive valid session then swap player, signer, game, tournament, wager, expiry, and quota values one at a time; expect rejection without partial value movement.

Deployment handoff checklist

Owner	Required evidence before external audit sign-off
Engineering	Exact commit hash, reproducible build command/features, BPF/SBF artifact hash, IDL, dependency lockfile.
DevOps	Target cluster, deployed program address, ProgramData address, upgrade authority and key-custody model.
MagicBlock integration	Pinned ER SDK/API versions, MagicBlock program IDs, router URL, validator/region policy, live delegation/undelegation transaction examples.
Treasury	Treasury authority custody, country-vault addresses, host treasury configuration, payout destinations, token mints.
Governance	Dispute/VPS/Link/KYC key custody, rotation/recovery policy, incident response.
QA	Passing full program tests plus live devnet ER lifecycle test, exact transaction logs, expected state snapshots before/after settlement.
Security	Known findings/accepted risks register, dependency CVEs, upgrade policy, emergency recovery runbook.

Code reference index

Topic	Pinned source
Program entrypoint / instruction inventory	[programs/xfchess-game/src/lib.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/lib.rs)
PDA seeds / authorities / constants	[programs/xfchess-game/src/constants.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/constants.rs)
Game account layout	[programs/xfchess-game/src/state/game.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/game.rs)
Tournament account layout	[programs/xfchess-game/src/state/tournament.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/tournament.rs)
PlayerProfile layout	[programs/xfchess-game/src/state/player_profile.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/player_profile.rs)

Topic	Pinned source
TournamentMatch layout	[programs/xfchess-game/src/state/tournament_match.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/tournament_match.rs)
Sessions	[programs/xfchess-game/src/state/player_session.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/player_session.rs); [programs/xfchess-game/src/state/global_session.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/global_session.rs); [programs/xfchess-game/src/state/tournament_session.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/tournament_session.rs)
Game creation/join/finalize/cancel	[programs/xfchess-game/src/game_ix/create.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/game_ix/create.rs); [programs/xfchess-game/src/game_ix/join.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/game_ix/join.rs); [programs/xfchess-game/src/game_ix/finalize.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/game_ix/finalize.rs); [programs/xfchess-game/src/game_ix/cancel.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/game_ix/cancel.rs)
Move validation	[programs/xfchess-game/src/moves_ix/apply.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/moves_ix/apply.rs)
Terminal rules	[programs/xfchess-game/src/lifecycle/terminal.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/lifecycle/terminal.rs)
Settlement	[programs/xfchess-game/src/lifecycle/settlement.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/lifecycle/settlement.rs)
Delegation boundary	[MAGICBLOCK.md](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/MAGICBLOCK.md)
Threat model	[docs/THREAT_MODEL.md](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/docs/THREAT_MODEL.md)
Dependency/test surface	[programs/xfchess-game/Cargo.toml](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/Cargo.toml)
Cluster/program mapping	[Anchor.toml](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/Anchor.toml)
Dynamic configuration state	[programs/xfchess-game/src/state/config.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/state/config.rs)
Dynamic configuration instructions	[programs/xfchess-game/src/account_ix/config_ix.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/account_ix/config_ix.rs)
Config integration tests	[programs/xfchess-game/tests/config_tests.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/tests/config_tests.rs)

Topic	Pinned source
Tournament stack remediation	[programs/xfchess-game/src/tournament_ix/lifecycle/cancel.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/tournament_ix/lifecycle/cancel.rs)
On-chain move validation	[programs/xfchess-game/src/moves_ix/apply.rs](https://github.com/trilltino/XFChess/blob/d7063b6fc1f9a29bcea9c5a3502ea5e714ec21fd/programs/xfchess-game/src/moves_ix/apply.rs)

Appendix A - Exact constants relevant to audit

LEGACY-VERSUS-CONFIGURED VALUES

The following values remain compiled constants in `constants.rs` at the current commit. `ProgramConfig` stores mutable equivalents for selected fields, but auditors should verify which call sites actually consume them before treating the config account as the authoritative source.

Constant	Value
MAX_WAGER_AMOUNT	10,000,000,000 lamports (10 SOL)
MAX_PLATFORM_FEE_LAMPORTS	1,000,000,000 lamports (1 SOL)
MIN_WAGER_LAMPORTS	1,000,000 lamports
CREATE_GAME_COST	5,000 lamports
JOIN_GAME_COST	5,000 lamports
DELEGATE_COST	5,000 lamports
UNDELEGATE_COST	5,000 lamports
RECORD_RESULT_COST	5,000 lamports per move
ER_SESSION_FEE_LAMPORTS	300,000 lamports
ER_COMMIT_FREQUENCY_MS	30,000 ms
CLAIM_PRIZE_COST	5,000 lamports
DISPUTE_RESOLUTION_COST_LAMPORTS	10,000 lamports
DISPUTE_BOND_LAMPORTS	10,000,000 lamports
ELO_FEE_LAMPORTS	5,000 lamports
DISPUTE_TTL_SECS	604,800 seconds / 7 days
MAX_SESSION_DURATION_SECS	2,592,000 seconds / 30 days

Constant	Value
WITHDRAW_EXPIRY_SECONDS	86,400 seconds / 24 hours
GAME_SESSION_EXPIRY_SECONDS	7,200 seconds / 2 hours
CRANK_MAX_SLOT_DELAY	300 slots
CRANK_MAX_SECONDS_EARLY	60 seconds

Appendix B - Review questions for the audit kickoff

- Is the deployed devnet program intended to be upgradeable during audit? If yes, which upgrade authority should the auditor model? If no, when will the deployment be made immutable?
- Will production use the same authority public keys as the pinned devnet build? The source explicitly expects mainnet rotation and will refuse placeholder keys.
- Which wager-token path is considered production-supported? The Game state supports `wager\_token`, but the core examples above describe the SOL path because it is directly visible in the pinned create/join/finalize flow.
- Which tournament payout models (LumpSum, StreamingLinear, StreamingCliff) are enabled in production, and are vesting accounts/claims part of the audit scope?
- Which MagicBlock validator/router endpoints are contractual production infrastructure rather than interchangeable devnet services?
- What key-management threshold (multisig, timelock, HSM, hardware signer) will protect VPS, dispute, treasury, Link, KYC, and upgrade authorities?

Document conclusion: The audit surface is best understood as four coupled security domains: (1) chess consensus/state transition, (2) delegated execution/persistence, (3) escrow/settlement conservation, and (4) privileged governance/session authorization. A credible audit should test domain boundaries and cross-domain transitions, not only individual instructions.